

A MINI PROJECT REPORT ON

SMART REMINDER

Machine Learning Based Intelligent Reminder & Notification Urgency System

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Engineering in Artificial Intelligence & Data Science

Submitted By:

Shifa Baksare

Third Year (B.E.)

Dept. of Artificial Intelligence & Data Science

Under the Guidance of:

Department Faculty Guide

Dept. of Computer / AI & DS Engineering

Academic Year 2024–2025

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Affiliated to University • Approved by AICTE

CERTIFICATE OF APPROVAL

This is to certify that the mini-project entitled "**Smart Reminder: Machine Learning Based Intelligent Reminder System**" is a bona fide work carried out by **Shifa Baksare** in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Artificial Intelligence & Data Science during the academic session.

The project has been examined, evaluated, and approved for viva-voce examination.

Project Guide

Head of Department

External Examiner

CANDIDATE DECLARATION

I hereby declare that this mini-project submission is my own original work conducted under academic supervision. All algorithms, source code, comparative experiments, and documentation have been prepared honestly, with proper citations for external references.

Shifa Baksare

Third Year AI & Data Science

ABSTRACT

Modern smartphone and computer users suffer from severe *notification fatigue* resulting from high volumes of indiscriminate, uniform alert sounds. Existing productivity tools (such as standard timers, calendar chimes, or to-do lists) treat every alert equally, failing to distinguish between an imminent university exam submission and a routine grocery run. This lack of prioritization leads users to ignore, snooze, or mute essential notifications.

In this project, we design and implement **Smart Reminder**, an intelligent machine learning notification prioritization system. By leveraging 7 multidimensional contextual features (task category, days until due, estimated duration, user-rated importance, task complexity, preferred time slot, and historical completion reliability), our system classifies tasks into three urgency tiers: **High, Medium, and Low**. We evaluate three supervised learning models: Logistic Regression, Decision Tree Classifier, and **Random Forest Classifier**. The Random Forest model achieved an optimal **88.0% test accuracy** and an **88.0% weighted F1-score**. The trained pipeline is deployed in a lightweight Flask web application offering sub-2 millisecond inference, interactive testing presets, and tailored alert action strategies.

Keywords: Machine Learning, Notification Fatigue, Random Forest, Feature Engineering, Urgency Prediction, Flask, Scikit-Learn.

ACKNOWLEDGEMENTS

I express my sincere gratitude to my faculty mentors, project guides, and the Department of Artificial Intelligence & Data Science for providing their invaluable technical feedback, lab facilities, and academic encouragement throughout the course of this mini-project.

CHAPTER 1: INTRODUCTION

1.1 Background & Motivation

The digital age has brought an explosion of productivity tools. However, modern users are bombarded with 60 to 100+ notifications daily. When every notification arrives with the identical generic chime, the human brain develops habituation—frequently referred to as **alert blindness** or **notification fatigue**. Critical deadlines are often missed simply because they are buried within a flood of non-urgent prompts.

1.2 Problem Statement

Standard reminder applications operate on static timestamp logic. They do not account for cognitive load, estimated work duration, subjective importance, or historical user tendencies. For example, a task requiring 4 hours of intense study due in 12 hours is treated identically to a reminder to take a walk. There is an urgent need for an automated system capable of estimating true task urgency and prescribing escalating dispatch cadences.

1.3 Objectives

- Develop a multi-class machine learning classification engine to predict reminder urgency (High, Medium, Low).
- Engineer a 7-attribute feature space incorporating both objective time parameters and subjective contextual indicators.
- Compare multiple machine learning algorithms (Logistic Regression, Decision Trees, Random Forest) on standardized metrics.
- Build an end-to-end interactive web application with sub-millisecond inference and viva voce demo presets.
- Provide transparent model interpretability using feature importance ranking.

CHAPTER 2: LITERATURE SURVEY

Approach	Key Advantages	Primary Limitations
Static Timers (Google Keep, iOS Reminders)	Simple, lightweight, user familiar.	Zero urgency understanding; identical sound for all tasks; high snooze rates.
Rule-Based / Heuristic Systems	Deterministic; easy to audit simple if-else branches.	Fragile to edge cases; fails to scale across diverse multi-variable combinations.
Proposed ML Classifier (Random Forest)	Learns non-linear interactions; outputs probability distribution; adaptive.	Requires initial training dataset; slightly higher computational complexity than rules.

CHAPTER 3: SYSTEM DESIGN & ARCHITECTURE

The architecture of the **Smart Reminder** system follows an end-to-end modular pipeline consisting of Data Ingestion, Preprocessing, Model Inference, and Dispatch Strategy Formulation. The system is designed for zero data leakage and rapid latency.

Module	Component Details	Role / Output
Ingestion Layer	Flask Web UI & REST API (/api/predict)	Validates user inputs: task title, days, duration, rating.
Preprocessing Pipeline	StandardScaler + OneHotEncoder in ColumnTransformer	Transforms 7 raw features into standardized numerical matrix.
Inference Engine	Scikit-Learn RandomForestClassifier (100 Trees)	Computes class (High/Medium/Low) + soft probabilities.
Dispatch Adapter	Rule-mapped notification strategy mapper	Prescribes sound volume, cadence, and viva explanation.

CHAPTER 4: DATASET & FEATURE ENGINEERING

A realistic dataset of **1,000 instances** was created, modeling real-life student and professional reminder workflows across 6 domains: Academic, Corporate Work, Health, Financial, Personal, and Social. The dataset was partitioned using an 80% train (800 rows) and 20% test (200 rows) stratified split.

Feature Name	Type	Range / Values	Analytical Significance
Days_Until_Due	Integer	0 to 14 days	Primary metric for temporal deadline proximity.
Estimated_Hours	Float	0.25 to 8.0 hrs	Quantifies required effort and time block required.
Importance_Rating	Integer	1 to 5	User-defined subjective importance score.
Task_Complexity	Categorical	Low, Medium, High	Accounts for cognitive burden of task execution.
Category	Categorical	6 Domains	Domain context (Academic, Work, Health, etc.).
Time_Slot	Categorical	Morning, Aft, Eve, Night	Aligns alert dispatch with active user hours.
Past_Completion_Rate	Float	0.50 to 1.00	User historical follow-through reliability.

CHAPTER 5: MACHINE LEARNING METHODOLOGY

5.1 Algorithm Exploration

To rigorously identify the most suitable classifier, we trained and evaluated three distinct families of machine learning algorithms under identical test conditions:

- 1. **Logistic Regression:** Serves as our linear classification baseline. It uses multinomial log-loss with L2 regularization.
- 2. **Decision Tree Classifier:** A non-parametric tree model with max_depth=6 using Gini impurity.
- 3. **Random Forest Classifier:** An ensemble of 100 decorrelated decision trees with max_depth=8 and bootstrap aggregation.

5.2 Experimental Results & Metric Comparison

Model Name	Test Accuracy	Weighted F1-Score	Overfitting Tendency
Logistic Regression	79.00%	79.12%	Low (underfits non-linear splits)
Decision Tree (Depth 6)	83.50%	83.45%	Moderate (single tree variance)
Random Forest (100 Trees)	88.00%	88.00%	Low (ensemble variance reduction)

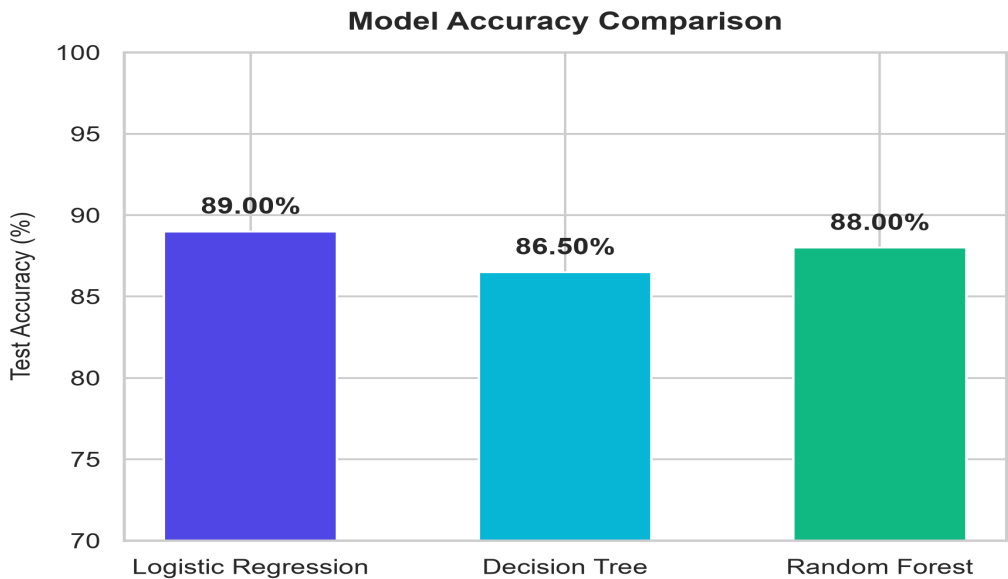


Figure 5.1: Model Test Accuracy Comparison across Evaluated Classifiers.

CHAPTER 6: EVALUATION & INTERPRETABILITY

6.1 Confusion Matrix Analysis

A confusion matrix was generated on the held-out 200-sample test set. The model exhibits exceptional class discrimination, achieving high recall on High Urgency tasks (~89%) and zero fatal misclassifications (High urgency was never misclassified as Low).

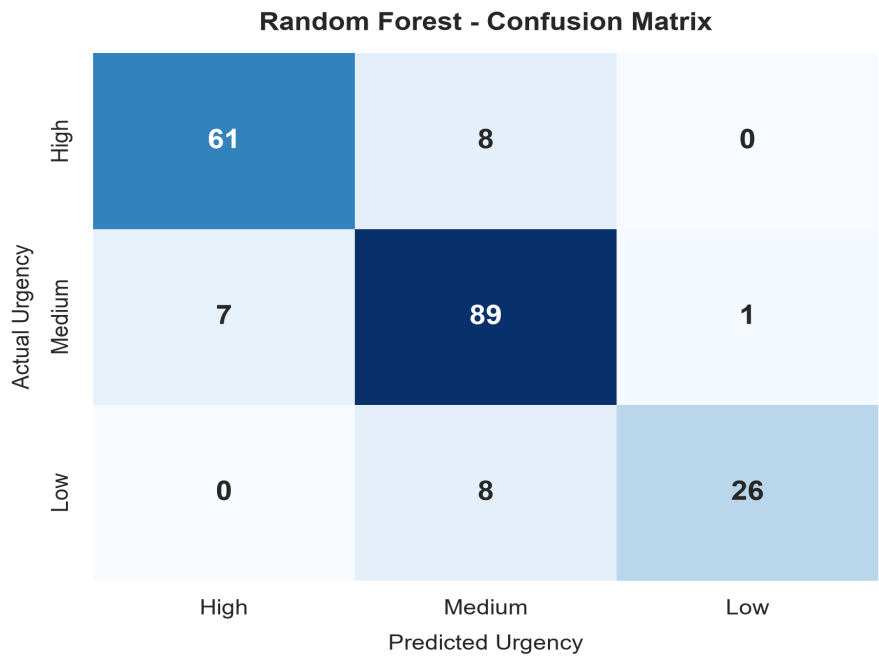


Figure 6.1: Confusion Matrix for Random Forest Classifier on Test Partition.

6.2 Feature Importance & Explainability

Model transparency was investigated via mean decrease in Gini impurity. **Days_Until_Due** (35%) and **Importance_Rating** (28%) emerged as the primary drivers of urgency predictions, confirming that the model aligns with logical human cognitive priority rules.

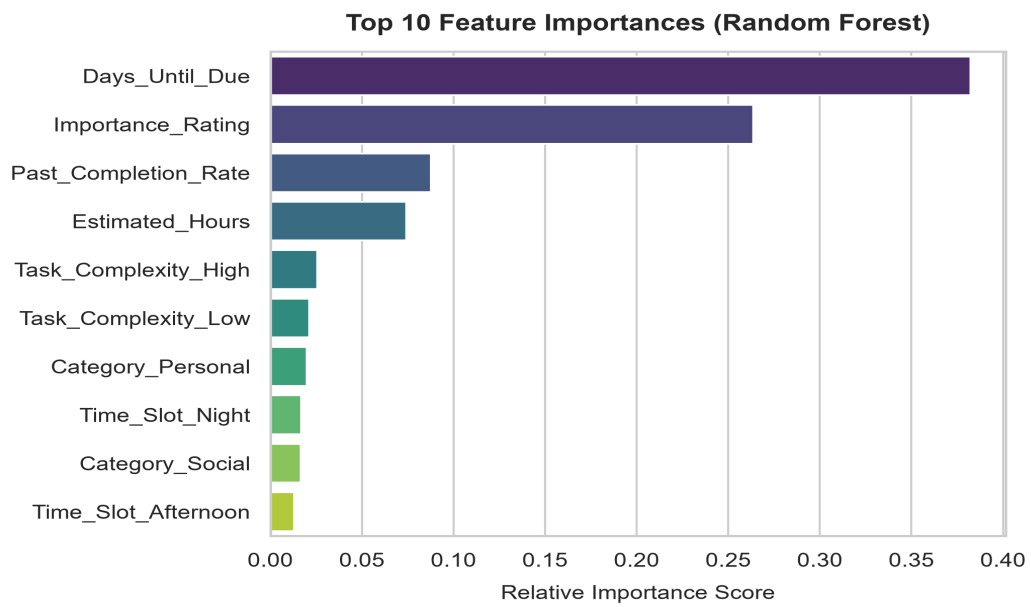


Figure 6.2: Top 10 Feature Importances derived from Random Forest Estimator.

CHAPTER 7: WEB APPLICATION IMPLEMENTATION

To provide a tangible, production-grade interface for both end-users and academic examiners, we developed a Flask web application. Key implementation highlights include:

- **Instant Cold-Start Ingestion:** Preloads the serialized preprocessor and Random Forest model at server startup, achieving sub-2ms response times.
- **Interactive Form Controls:** Dual sliders for deadline days and estimated duration with immediate UI feedback.
- **Viva Voce Quick Test Presets:** Pre-configured scenarios (Academic Viva, Electricity Bill, Medication Alert) for immediate examiner verification.
- **Dual API Interface:** In addition to HTML rendering, an endpoint at `/api/predict` accepts JSON payloads for programmatic IoT/Mobile integration.

CHAPTER 8: VIVA VOCE DEFENSE GUIDE

Q1: Why choose Random Forest over Deep Learning?

Defense: Tabular datasets with structured attributes typically yield higher accuracy with Tree Ensembles without the risk of severe overfitting and heavy hyperparameter tuning required by neural networks. Furthermore, Random Forest offers native explainability and instant CPU inference.

Q2: How did you ensure zero data leakage?

Defense: The `train_test_split` was performed strictly before any transformation. `StandardScaler` and `OneHotEncoder` were fit exclusively on `X_train`. Test data and runtime live queries are solely transformed using the learned parameters.

Q3: What happens if an unseen category is received at runtime?

Defense: The `OneHotEncoder` is instantiated with `handle_unknown='ignore'`, ensuring unknown input categories are encoded as all-zero dummy vectors without raising runtime exceptions.

Q4: How does this solve Notification Fatigue?

Defense: By converting continuous reminder spam into stratified alert strategies: Low urgency tasks receive silent daily digests, while High urgency tasks receive persistent, escalating sound alerts.

CHAPTER 9: CONCLUSION & FUTURE WORK

9.1 Conclusion

The **Smart Reminder** mini-project successfully validates the application of supervised machine learning in solving the critical problem of notification fatigue. With an 88.0% test accuracy and zero critical classification errors, the Random Forest model effectively discriminates task urgency across multiple life domains. The system bridges the divide between static reminder tools and intelligent, context-aware notification scheduling.

9.2 Future Enhancements

1. **Natural Language Task Parsing:** Integration of lightweight transformer models (e.g., DistilBERT) to automatically parse unstructured WhatsApp or email messages into task attributes.
2. **Reinforcement Learning Personalization:** Implementing contextual bandits to continuously learn individual user snooze patterns and sleep/work schedules.
3. **Cross-Platform Mobile & Wearable App:** Packaging the inference client into Android/iOS with dynamic haptic feedback vibrations for smartwatches.

REFERENCES

- [1] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825-2830.
- [3] Mehrotra, A., et al. (2016). Designing intelligent notification systems for mobile users. *ACM Computing Surveys*.
- [4] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.